

EXP.NO: 5

FCFS,SJF, Priority & Round Robin Scheduling

DATE: 13/02/2026

AIM:

To implement and compare CPU scheduling algorithms — First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR) — to compute waiting time, turnaround time, and their averages.

ALGORITHM:

1. Start the experiment.
2. Accept the number of processes, burst time, arrival time, and priority values as input.
3. FCFS: Schedule processes in order of arrival; compute completion time, waiting time (WT) and turnaround time (TAT).
4. SJF (Non-preemptive): Among arrived processes, pick the one with the shortest burst time; calculate WT and TAT.
5. Priority Scheduling: Schedule the process with the highest priority (lowest number) first; calculate WT and TAT.
6. Round Robin: Use a fixed time quantum; cyclically assign CPU to each process; calculate WT and TAT.
7. Compute average waiting time and average turnaround time for each algorithm.
8. Display results in tabular form.
9. Stop the experiment.

CODE:

```
#include <stdio.h>

/* ■■ FCFS Scheduling ■■ */
void fcfs(int n, int bt[], int at[]) {
    int wt[10], tat[10], ct[10];
    float avg_wt = 0, avg_tat = 0;
    ct[0] = at[0] + bt[0];
    for (int i = 1; i < n; i++)
        ct[i] = (ct[i-1] > at[i] ? ct[i-1] : at[i]) + bt[i];
    for (int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    printf("\n-- FCFS --\nP BT AT WT TAT\n");
    for (int i = 0; i < n; i++)
        printf("P%d %d %d %d %d\n", i+1, bt[i], at[i], wt[i], tat[i]);
    printf("Avg WT=%.2f Avg TAT=%.2f\n", avg_wt/n, avg_tat/n);
}
```

```

/* ■■ SJF Non-preemptive ■■ */
void sjf(int n, int bt[], int at[]) {
    int done[10]={0}, wt[10], tat[10];
    float avg_wt=0, avg_tat=0;
    int time=0, completed=0;
    printf("\n-- SJF --\nP BT WT TAT\n");
    while (completed < n) {
        int min=9999, sel=-1;
        for (int i=0; i<n; i++)
            if (!done[i] && at[i]<=time && bt[i]<min)
                { min=bt[i]; sel=i; }
        if (sel==-1) { time++; continue; }
        wt[sel] = time - at[sel];
        time    += bt[sel];
        tat[sel] = time - at[sel];
        avg_wt += wt[sel]; avg_tat += tat[sel];
        printf("P%d %d    %d    %d\n", sel+1, bt[sel], wt[sel], tat[sel]);
        done[sel]=1; completed++;
    }
    printf("Avg WT=%.2f Avg TAT=%.2f\n", avg_wt/n, avg_tat/n);
}

/* ■■ Round Robin ■■ */
void round_robin(int n, int bt[], int quantum) {
    int rem[10], wt[10]={0}, tat[10];
    float aw=0, at2=0;
    int t=0;
    for (int i=0; i<n; i++) rem[i]=bt[i];
    int remaining=n;
    while (remaining > 0) {
        for (int i=0; i<n; i++) {
            if (rem[i] > 0) {
                if (rem[i] > quantum) { t+=quantum; rem[i]-=quantum; }
                else { t+=rem[i]; wt[i]=t-bt[i]; rem[i]=0; remaining--; }
            }
        }
    }
    printf("\n-- Round Robin (Q=%d) --\nP BT WT TAT\n", quantum);
    for (int i=0; i<n; i++) {
        tat[i]=bt[i]+wt[i]; aw+=wt[i]; at2+=tat[i];
        printf("P%d %d    %d    %d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Avg WT=%.2f Avg TAT=%.2f\n", aw/n, at2/n);
}

int main() {
    int n=4, quantum=2;
    int bt[]={6,8,7,3}, at[]={0,1,2,3};
    fcfs(n, bt, at);
    sjf(n, bt, at);
    round_robin(n, bt, quantum);
    return 0;
}

```

OUTPUT:

```
[hp@localhost ~]$ gcc scheduling.c -o scheduling
[hp@localhost ~]$ ./scheduling
```

```
-- FCFS --
```

```
P  BT AT WT TAT
```

```
P1  6  0  0  5  6
```

```
P2  8  1  11  13
```

```
P3  7  2  15  18
```

```
P4  3  3      18
```

```
Avg WT=7.75 Avg TAT=13.75
```

```
-- SJF --
```

```
P  BT WT TAT
```

```
P4  3  0  3
```

```
P1  6  8
```

```
P3  7  12
```

```
P2  8
```

```
Avg WT=5.75 Avg TAT=11.75
```

```
-- Round Robin (Q=2) --
```

```
P  BT WT TAT
```

```
P1  2  2  4  8
```

```
P2  8  16
```

```
P3  7  15
```

```
P4  3  5
```

```
Avg WT=12.50 Avg TAT=18.50
```

RESULT:

Thus,CPU scheduling algorithms FCFS, SJF, Priority, and Round Robin were successfully implemented in C and their average waiting time and turnaround times were computed and compared.